

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Perú, Decana de América

Facultad de Ingeniería de Sistemas e Informática



CURSO: Lenguajes y Compiladores

DOCENTE: Prudencio Vidal, Javier Antonio

GRUPO: 5

INTEGRANTES:

Arboleda Sanchez, Ericka Sofia	24200047
Badillo Castillo, Edwin Piero	24200150
Ccoicca Fernandez, Adrian	24200154
Huamanchahua Huayanay, Oscar	24200161
Maque Espinoza, Steveen Dennys	24200059
Reynoso Castillo, José Luis	24200169
Zarate Vilchez, Axhell Jesus	24200177

Cronograma

A. Semana 2

- a. Definición formal del lenguaje Dummy
- b. Diseño de sintaxis (estilo tipo C simplificado)
- c. Definición de palabras reservadas (inicio, cuando, sino, loop, imprimir, retornar)
- d. Definición de tipos (num, dec, text, booleano)
- e. Estructura de programa y ejemplos funcionales

B. Semana 3 (ya realizado – Fase 1 avanzada)

- a. Definición completa de tokens
- b. Expresiones regulares por token
- c. Construcción de AFN y conversión a AFD
- d. Tablas de transición
- e. Validación teórica del análisis léxico

C. Semana 4 (Fase 1 – implementación real)

- a. Implementación del lexer en Flex
- b. Retorno de tokens (no printf)
- c. Reconocimiento de keywords, IDs, literales y operadores
- d. Manejo de espacios y comentarios
- e. Primeras pruebas de ejecución

D. Semana 5 (Fase 1 – cierre)

- a. Manejo de errores léxicos (caracteres inválidos, tokens mal formados)
- b. Corrección de reglas en Flex
- c. Pruebas completas con programas Dummy

- d. Compilación y ejecución correcta en tu máquina
- e. Lexer listo para integrarse con parser

E. Semana 6 (Fase 2 – inicio)

- a. Definición de la gramática formal (BNF/EBNF)
- b. Reglas para: declaraciones, asignaciones, condicionales (cuando/sino)
- c. Reglas para bucles (loop)
- d. Reglas para funciones y retornar
- e. Eliminación de ambigüedad y recursión izquierda

F. Semana 7 (Fase 2 – parser)

- a. Implementación del parser en Bison/Yacc
- b. Definición de tokens compartidos con Flex
- c. Producciones por cada no terminal
- d. Pruebas iniciales del parser
- e. Detección de errores sintácticos

G. Semana 8 (Fase 2 – integración)

- a. Conexión real lexer + parser (yylex retorna tokens)
- b. Eliminación de prints del lexer
- c. Validación de parsing completo
- d. Manejo básico de errores sintácticos
- e. Ajustes de gramática

H. Semana 9 (Fase 2 – AST)

- a. Diseño del Árbol de Sintaxis Abstracta (AST)
- b. Definición de nodos (asignación, condicional, loop, función)
- c. Construcción del AST en Bison
- d. Pruebas con programas completos

- e. Validación de estructura del árbol
- I. Semana 10 (Fase 2 – cierre + robustez)
 - a. Manejo de precedencia de operadores
 - b. Validación de expresiones complejas
 - c. Pruebas con anidamiento (loop dentro de cuando, etc.)
 - d. Optimización del parser
 - e. Parser + AST estables
- J. Semana 11 (Fase 3 – semántico)
 - a. Diseño de tabla de símbolos
 - b. Registro de variables (nombre, tipo)
 - c. Manejo de scope (bloques { })
 - d. Registro de funciones
 - e. Integración con AST
- K. Semana 12 (Fase 3 – validación)
 - a. Verificación de tipos (num vs text vs dec)
 - b. Validación de variables declaradas
 - c. Detección de variables no declaradas
 - d. Validación de uso correcto de funciones
 - e. Errores semánticos claros
- L. Semana 13 (Fase 4 – código)
 - a. Diseño de código intermedio (3 direcciones o similar)
 - b. Generación desde el AST
 - c. Traducción a ensamblador Z80
 - d. Mapeo de variables a memoria
 - e. Traducción de:

- i. Asignaciones
 - ii. Operaciones aritméticas
 - iii. Condicionales (saltos)
 - iv. Bucles (loop $\rightarrow$ etiquetas + jumps)
- f. Pruebas completas del compilador
- g. Optimización básica (opcional)
- h. Integración final (lexer + parser + semántico + código)
- i. Documentación técnica final